Travaux Pratiques de Programmation Android

TP n° 2

Les objectifs de ce TP sont :
- Présenté le cycle de vie d'une Activity avec manipulation des callbacks.
- Développement d'une application Android multi widgets.
- Familiarisation avec les values, dimensions, colors, internationalisation

Créez un nouveau projet Android nommé GLSI3_votre_groupe, puis complétez celui-ci avec les étapes successives suivantes :

Exercice 1 : Cycle de vie d'une application Android et les *callbacks*

Une activité n'est pas linéaire, elle est soumise à plusieurs évènements. Chaque évènement est représenté par une méthode. La figure suivante illustre le cycle de vie d'une application Android :

- **onCreate():** Cette méthode est appelée à la création d'une activité (Activity). Elle permet de l'initialiser. C'est ici que l'interface graphique est spécifiée.
- **onStart():** Cette méthode est appelée quand l'application est démarrée.
- **onResume():** Cette méthode est appelée quand l'application passe (ou repasse) en avant plan.
- **onPause**(): Appelée quand l'application passe en arrière-plan et qu'une autre application se met devant.
- **onStop**(): Appelée quand l'application n'est plus visible.
- **onRestart()**: Appelée quand l'application redevient visible.
- **onDestroy():** Appelée quand votre application est fermée par le système à cause d'un manque de ressources, ou par l'utilisateur à l'utilisation d'un finish(). Toutes les données non sauvegardées sont perdues.

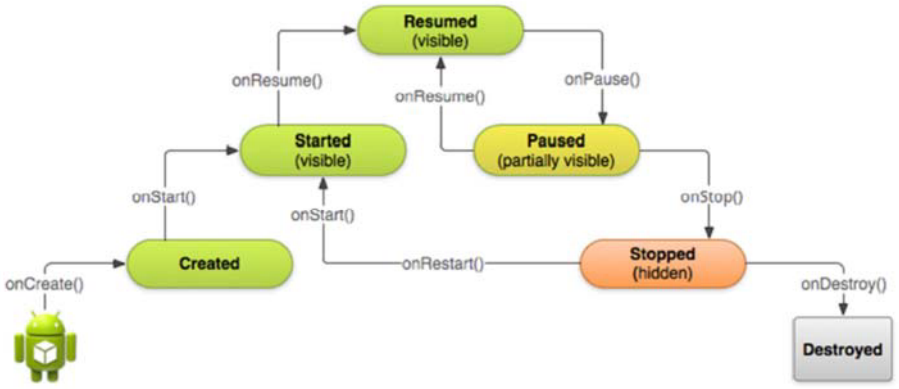


Figure 1 Schéma simplifié du cycle de vie de l'activité

Pour générer ces méthodes, commencer à taper le nom de la méthode dans le fichier de code, puis taper sur les touches *Ctrl+Espace*

Il existe plusieurs situations dans lesquelles une activité transite entre les différents états qui sont illustrés dans la figure 1. Cependant, seulement trois de ces états peuvent être stables. C'est-à-dire, l'activité peut exister dans l'un de ces trois états seulement pendant une période prolongée : *Resumed, Stoped, Paused.*

Les autres états *Created* et *Started* sont transitoires et le système passe rapidement à l'état suivant en appelant la méthode de *Callback* de cycle de vie suivante. C'est-à-dire, après que le système appelle *onCreate()*, il appelle rapidement *onStart()*, qui est rapidement suivi par *onResume()*.

Travail demandé : Traitement des Callbacks

Générer les méthodes correspondant à chacun des évènements listés dans le cycle de vie. Nous désirons voir le comportement de l'activité pour chacun de ces évènements. Pour cela, chaque méthode va afficher son nom à travers un Toast.

Exercice 2 : Développement d'une application Android multi-widgets



1. Créer le formulaire de l'application en XML, en respectant les contraintes suivantes :
   - Les champs **Nom** et **Prénom** doivent accepter uniquement du texte.
   - Les champs **jj** et **mm** (pour le jour et le mois de la date de naissance) doivent accepter uniquement deux chiffres.
   - Le champ **aaaa** (pour l'année de la date de naissance) doit accepter uniquement quatres chiffres.
   - Les filières **Phy3** et **Chimi3** comportent chacune deux groupes, tandis que la filière **GLSI3** en comporte trois.
2. Utiliser le fichier **strings.xml** pour stocker tous les titres, sous-titres, et hints, afin de faciliter l'internationalisation de l'application.
3. Utiliser le fichier **colors.xml** pour standardiser les couleurs des widgets : par exemple, bleu foncé pour les sous-titres et bleu pour les champs de saisie.
4. Ajouter un fichier **dimensions.xml** pour uniformiser les tailles des éléments : par exemple, **25sp** pour les sous-titres, **20sp** pour le texte des boutons, **40sp** pour le titre, etc.
5. Créer des dictionnaires pour les trois langues : Arabe, Anglais, et Français.
6. Programmer les événements de clic pour les boutons comme suit :
   - Le bouton **Valider** affiche les informations saisies dans la zone d'affichage.
   - Le bouton **RAZ** réinitialise les champs de saisie.
   - Le bouton **Quitter** ferme l'application.